Iron Stats: Continuous Biomechanical Tension Modeling, Physiological Fatigue Dynamics, and Multi-Objective Mesocycle Optimization

Lorenzo Zambardi
https://github.com/Lorenzo1901/Iron-Stats

Technical Whitepaper — November 2026

Abstract

Conventional resistance-training analytics quantify training stress via coarse discrete scalars—raw set counts, repetition totals, and mechanical tonnage ($L \times R$). Such aggregates implicitly assume uniform mechanical tension across the entire Range of Motion (ROM), identical motor-unit recruitment regardless of proximity to muscular failure, and linear fatigue accumulation across repetitions. **Iron Stats** replaces these scalar heuristics with a unified computational model spanning three layers: (1) *continuous biomechanical tension fields*, parameterizing per-muscle resistance profiles over normalized ROM $x \in [0, 1]$ via area-normalized parametric cubic Bézier curves with strictly monotone ROM coordinate $x(t)$; (2) *physiological load and fatigue dynamics*, modeling High-Threshold Motor Unit (HTMU) recruitment via an RPE-gated effective-repetition threshold, piecewise RIR-dependent concentric Time Under Tension (TUT) slowdown, and continuous exponential central nervous system (CNS) fatigue; and (3) *constrained combinatorial optimization*, synthesizing balanced multi-day mesocycles via Monte Carlo scale calibration and scalarized Simulated Annealing. We present a numerical analysis comparing Lagrange interpolation, C^2 natural cubic splines, and Bernstein-basis cubic Bézier curves on empirical exercise profiles—explicitly separating boundary discontinuities from smooth interior interpolation errors—and prove why Bézier convex-hull containment and $\mathcal{O}(1)$ Bernstein evaluation eliminate non-physical negative tension while supporting exact 3-point Gauss–Legendre magnitude conservation.

1 Introduction

Quantitative hypertrophy programming traditionally reduces a resistance-training session to discrete scalar summaries:

$$V_{\text{naive}} = \sum_s R_s, \qquad T_{\text{naive}} = \sum_s L_s R_s, \tag{1}$$

where R_s and L_s denote repetitions and external load in set s. Despite widespread adoption, these scalars exhibit three structural deficiencies:

1. **Biomechanical Blindness:** Moment arms and external resistance torque vary continuously across joint angles. Dumbbell Lateral Raises (`LR`), for example, peak in resistance near full abduction ($x \to 0.9$), whereas stretch-biased movements peak near $x \to 0$. Scalar tonnage treats both identically.
2. **Effort Invariance:** Two sets of 10 repetitions—one stopped at Rate of Perceived Exertion $\text{RPE} = 5$ (5 Reps in Reserve, $\text{RIR} = 5$) and one pushed to concentric failure ($\text{RPE} = 10$)—yield identical V_{naive}, despite markedly different HTMU recruitment and velocity loss.
3. **Linear Fatigue Assumption:** Systemic CNS and local tissue fatigue grow nonlinearly as repetitions approach failure, accompanied by progressive concentric velocity decay.

To address these gaps, **Iron Stats** models every exercise over a normalized continuous ROM domain $x \in [0, 1]$, integrates per-repetition physiological slowdown and exponential fatigue, and formulates mesocycle synthesis as a constrained multi-objective combinatorial optimization problem. All equations reported herein match the reference implementation in `python_research/analyzer/` (`constants.py`, `models.py`, `metrics.py`, `parser.py`, `solver.py`).

2 Biomechanical Exercise Representation

In `models.py`, each exercise e is parameterized by a biomechanical tuple

$$\theta_e = \big(c_{\text{load}},\, f_{\text{base}},\, \mu_{\text{load}},\, q_{\text{load}},\, \text{is_isolation}\big), \tag{2}$$

where $c_{\text{load}} \geq 0$ denotes articular load efficiency, $f_{\text{base}} \geq 0$ is the intrinsic systemic fatigue coefficient, $(\mu_{\text{load}}, q_{\text{load}}) \in \mathbb{R}^+ \times \mathbb{R}$ maps nominal logged load L_{raw} to effective biomechanical load via

$$L = L_{\text{raw}} \cdot \mu_{\text{load}} + q_{\text{load}}, \tag{3}$$

and $\text{is_isolation} \in \{\text{False}, \text{True}\}$ flags single-joint movements. (`Exercise.__post_init__` validates $c_{\text{load}} \geq 0$, $f_{\text{base}} \geq 0$, and $\mu_{\text{load}} > 0$; across the 61-exercise JSON catalog, $c_{\text{load}} \in [0.01, 1.0]$

and $f_{\text{base}} \in [1, 10]$, while `ExerciseModal.jsx` and `App.jsx::buildValidatedExercise` bound UI inputs to $[0, 2]$ and $[0, 10]$, and `LR` specifies $\mu_{\text{load}} = 2.0$ to convert per-dumbbell weight into bilateral load.)

Each exercise targets an anatomical sub-muscle set $\mathcal{M}_e = \{m_1, \ldots, m_k\}$ with contribution magnitudes $\mathbf{M}_e = [M_{e,m_1}, \ldots, M_{e,m_k}]^\top$. Whenever $\mathcal{M}_e \neq \emptyset$, `Exercise.__post_init__` conserves total mechanical work attribution across muscle compartments via the partition-of-unity constraint

$$\sum_{m \in \mathcal{M}_e} M_{e,m} = 1.0, \tag{4}$$

returning re-normalized copies $M_{e,m} \leftarrow M_{e,m} / \sum_j M_{e,j}$ whenever $|\sum_m M_{e,m} - 1.0| > 10^{-12}$ (with $M_{e,m} \geq 0$, preserving floating-point partition of unity across the 61-exercise JSON catalog and `override:` directives).

3 Numerical Analysis of Tension Profiles

Representing muscle tension $y(x)$ across normalized ROM $x \in [0, 1]$ requires a functional basis that is parsimonious for interactive editing, smooth, numerically stable, and physically non-negative ($y(x) \geq 0$, as skeletal muscle cannot generate repulsive active tension). We evaluate three candidate numerical frameworks on the `LR` (*Lateral Deltoid*) reference profile defined in `defaultExercises.json` ($P_0 = (0.20, 0.00)$, $P_1 = (0.33, 0.30)$, $P_2 = (0.66, 0.80)$, $P_3 = (0.90, 1.00)$, $M = 0.90$).

The reference `LR` profile is $y(x) = 0$ for $x < x_0$ and $x > x_3$, with $y(x_3^-) = 1.0$ and $y(x_3^+) = 0.0$ (a jump discontinuity at x_3) and a C^0 slope corner at $x_0 = 0.20$ ($y(x_0^-) = y(x_0^+) = 0$, $y'(x_0^+) \approx 2.31$). Any continuous global interpolant (Lagrange or C^2 cubic spline) on $[0, 1]$ must therefore exhibit a Gibbs-like overshoot near x_3—and oscillatory ripples across $[0, x_0]$—independent of node spacing; on equispaced nodes, exponential Lebesgue amplification (the Runge effect) further magnifies boundary errors relative to clustered Chebyshev nodes. To isolate pure approximation error from the boundary discontinuities, each comparison below pairs the full domain $[0, 1]$ with a restricted smooth interior domain $[x_0+\delta, x_3-\delta] = [0.25, 0.85]$ ($\delta = 0.05$).

3.1 Lagrange Interpolation and Runge vs. Gibbs Effects

Given n sampled ROM-tension nodes $\{(x_i, y_i)\}_{i=1}^n$, the unique degree-$(n-1)$ global interpolating polynomial is

$$p_{n-1}(x) = \sum_{i=1}^{n} y_i \ell_i(x), \qquad \ell_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}. \tag{5}$$

On uniform nodes, solving in the monomial Vandermonde basis ($V\mathbf{a} = \mathbf{y}$, as used by `numpy.polyfit`) reaches condition number $\kappa(V) = 2.01 \times 10^6$ at $n = 9$, while the equispaced Lebesgue constant itself grows exponentially as $\Lambda_n \sim \frac{2^n}{e(n-1)\ln(n-1)}$ (whereas Vandermonde ill-conditioning can be avoided in evaluation via barycentric weights, Λ_n depends solely on the nodes and cannot be removed by basis choice; the companion figure script calls `numpy.polyfit` directly on $[0, 1]$ for the full-domain curves—with $\kappa(V) = 2.01 \times 10^6$ at $n = 9$ uniform and 1.02×10^8 at $N = 12$ CGL—and rescales $[0.25, 0.85] \to [0, 1]$ on the restricted domain). On the full domain $[0, 1]$, equispaced Lagrange interpolation ($n = 9$) yields L^∞ error 0.961 (evaluated on a dense grid bracketing $x_3^\pm$), dominated by the jump at $x_3 = 0.90$ and resting-zone ripples in $[0, 0.20]$. On the restricted smooth interior $[0.25, 0.85]$, however, the L^∞ error drops to 2.89×10^{-5} (Fig. 1).

Replacing equispaced nodes with Chebyshev–Gauss–Lobatto (CGL) nodes mapped to $[a, b]$,

$$x_k = a + \frac{b-a}{2}\left(1 - \cos\frac{k\pi}{N-1}\right), \qquad k = 0, \ldots, N-1, \tag{6}$$

suppresses the Runge phenomenon and reduces Lebesgue growth to $\Lambda_N = \frac{2}{\pi}\ln N + \mathcal{O}(1)$. On the smooth interior $[0.25, 0.85]$, CGL achieves $L^\infty = 2.07 \times 10^{-4}$ ($N = 6$) and 4.90×10^{-7} ($N = 12$). Yet on the full physical domain $[0, 1]$, global coupling across the discontinuity at x_3 still yields $L^\infty = 0.989$ ($N = 6$) and 0.805 ($N = 12$) with negative dips ($y(x) < 0$) in the resting zone $x \in [0, 0.2]$ (Fig. 2), making polynomial interpolation unsuitable for bounded-support exercise profiles.

3.2 C^2 Natural Cubic Splines

Piecewise natural cubic splines partition $[a, b]$ using N knots $a = \tilde{x}_0 < \tilde{x}_1 < \cdots < \tilde{x}_{N-1} = b$ into $N-1$ subintervals $[\tilde{x}_k, \tilde{x}_{k+1}]$, enforcing C^2 interior continuity and zero boundary curvature $S''(\tilde{x}_0) = S''(\tilde{x}_{N-1}) = 0$. Solving the tridiagonal system $A\mathbf{m} = \mathbf{d}$ via the Thomas algorithm takes $\mathcal{O}(N)$ operations and minimizes $\int_a^b [S''(x)]^2\, dx$.

On the restricted smooth interior $[0.25, 0.85]$, a uniform 6-knot natural C^2 spline attains $L^\infty = 2.46 \times 10^{-3}$ without oscillation. On the full domain $[0, 1]$, even with knots placed directly at the support boundaries ($N = 6$ non-uniform knots $\{0, 0.2, 0.45, 0.75, 0.9, 1\}$), C^2 stiffness produces non-physical negative undershoot ($y(x) < 0$) across the flat-to-ramp transition $x \in (0, 0.2)$ alongside a unit jump discrepancy ($L^\infty = 1.000$ at x_3^+) where the continuous interpolant passes through $(0.9, 1.0)$ (Fig. 3). Uniform knot refinement does not eliminate negative undershoot across the discontinuity: at $N = 10$ uniform knots, the natural C^2 spline still dips below zero ($\min y = -4.29 \times 10^{-4}$, $L^\infty = 0.942$), and denser uniform grids ($N \in [15, 50]$) exhibit Gibbs undershoots down to -0.13 past x_3. Eliminating undershoot requires non-uniform knots tightly bracketing the discontinuities or relaxing continuity to C^1 monotone piecewise

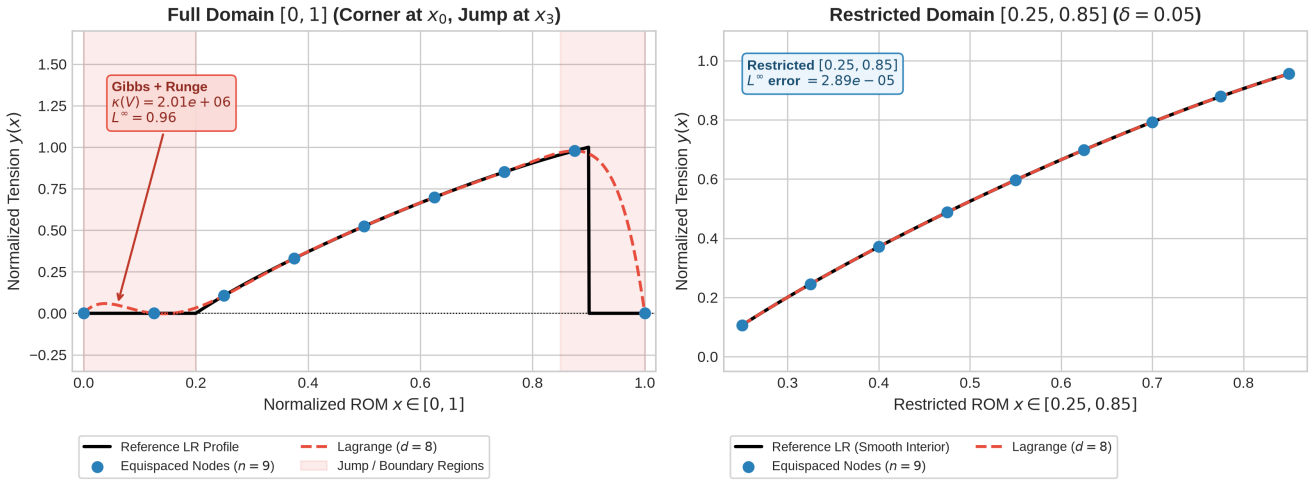


Figure 1: Lagrange interpolation ($n = 9$ equispaced nodes) on LR. Left: full domain $[0, 1]$, showing Gibbs overshoot at $x_3 = 0.90$ and resting-zone oscillations ($L^\infty = 0.961$, monomial $\kappa(V) = 2.01 \times 10^6$). Right: restricted domain $[0.25, 0.85]$ ($\delta = 0.05$), isolating smooth interior error ($L^\infty = 2.89 \times 10^{-5}$).

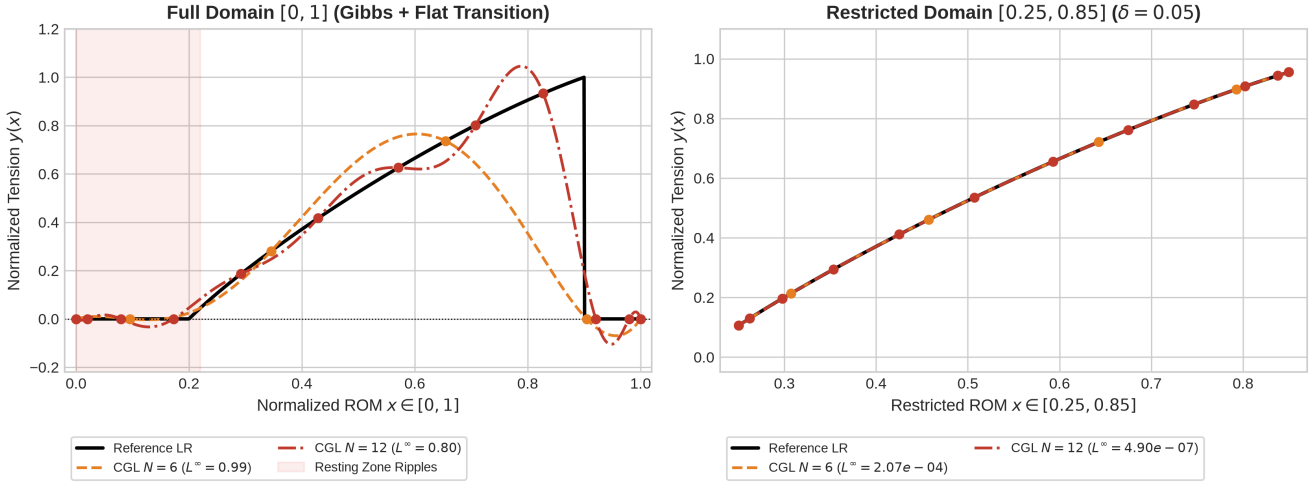


Figure 2: Chebyshev–Gauss–Lobatto interpolation ($N = 6, 12$) on LR. Left: full domain $[0, 1]$, showing Gibbs error at x_3 and negative ripples in $x \in [0, 0.2]$ ($L^\infty = 0.989$ for $N = 6$, 0.805 for $N = 12$). Right: restricted domain $[0.25, 0.85]$ ($L^\infty = 2.07 \times 10^{-4}$ and 4.90×10^{-7}).

cubic Hermite interpolation (PCHIP), sacrificing parameter parsimony or C^2 curvature (Fig. 4).

3.3 Cubic Bézier Curves, Monotonicity, and Bernstein Guarantees

In `BezierProfile` (`models.py`), each active tension profile over $[x_0, x_3] \subseteq [0, 1]$ is defined parametrically by four absolute 2D control points $\mathbf{P}_i = (x_i, y_i)^\top$ with $x_i \in [0, 1]$ and $y_i \geq 0$, $i \in \{0, 1, 2, 3\}$ (default catalog and interactive editor profiles lie in $[0, 1]^2$), in the cubic Bernstein basis (and $y(x) = 0$ for $x \notin [x_0, x_3]$):

$$\mathbf{P}(t) = \begin{pmatrix} x(t) \\ y(t) \end{pmatrix} = \sum_{i=0}^{3} \mathbf{P}_i B_{i,3}(t), \qquad t \in [0, 1], \tag{7}$$

where $B_{i,3}(t) = \binom{3}{i} t^i (1-t)^{3-i}$. Explicitly, both components share the same convex combination:

$$x(t) = (1-t)^3 x_0 + 3(1-t)^2 t\, x_1 + 3(1-t) t^2\, x_2 + t^3 x_3, \tag{8}$$

$$y(t) = (1-t)^3 y_0 + 3(1-t)^2 t\, y_1 + 3(1-t) t^2\, y_2 + t^3 y_3. \tag{9}$$

Theorem 1 (Convex Hull Non-Negativity). *For all $t \in [0, 1]$, the cubic Bernstein polynomials satisfy $B_{i,3}(t) \geq 0$ and form a partition of unity $\sum_{i=0}^{3} B_{i,3}(t) = \big((1-t) + t\big)^3 \equiv 1$. Consequently, $\mathbf{P}(t) \in \mathrm{Conv}(\{\mathbf{P}_0, \mathbf{P}_1, \mathbf{P}_2, \mathbf{P}_3\})$, and if $y_i \geq 0$ for all $i \in \{0, 1, 2, 3\}$, then $y(t) \geq 0$ for every $t \in [0, 1]$.*

Proof. Since $t \geq 0$ and $1 - t \geq 0$ on $[0, 1]$, each $B_{i,3}(t) \geq 0$. By the binomial theorem, $\sum_{i=0}^{3} \binom{3}{i} t^i (1-t)^{3-i} = 1$, so $y(t) = \sum_{i=0}^{3} B_{i,3}(t) y_i$ is a convex combination of $\{y_i\}$

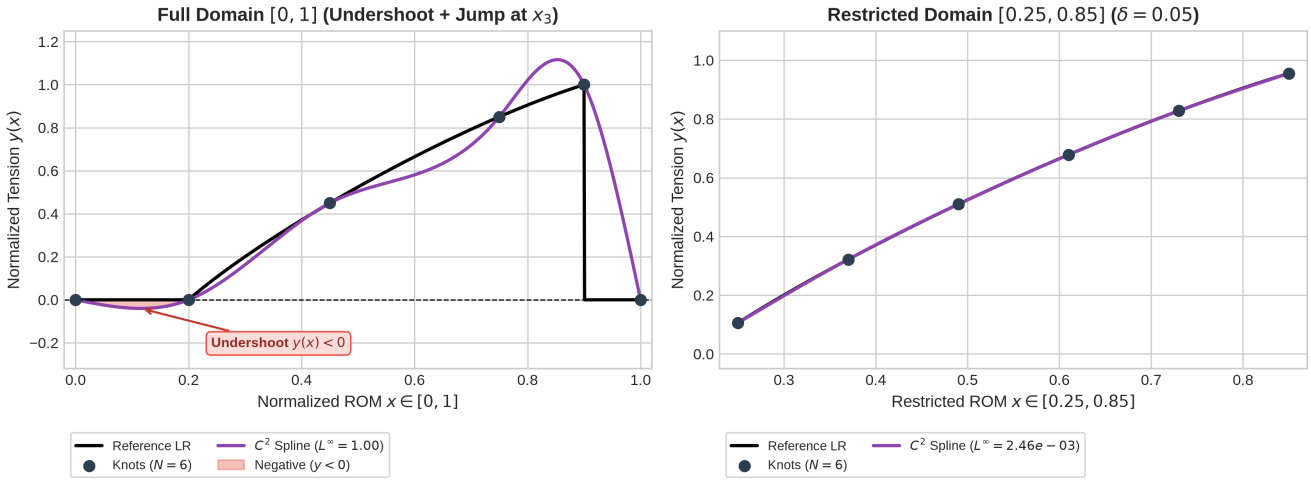


Figure 3: Natural C^2 cubic spline ($N = 6$ knots) on `LR`. Left: full domain $[0, 1]$ (non-uniform knots placing nodes at $x_0 = 0.2$ and $x_3 = 0.9$), exhibiting non-physical negative undershoot ($y(x) < 0$) in $(0, 0.2)$ and unit jump error at x_3^+ ($L^\infty = 1.000$). Right: restricted domain $[0.25, 0.85]$ with 6 uniform knots ($L^\infty = 2.46 \times 10^{-3}$).

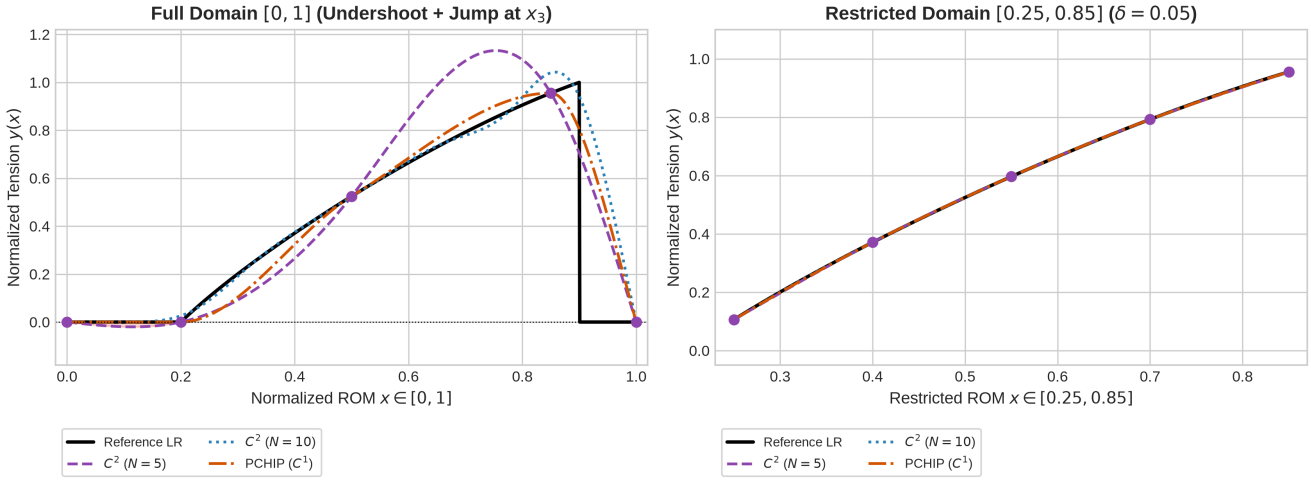


Figure 4: Spline trade-off on `LR`. Left: full domain $[0, 1]$ (non-uniform $N = 5$ knots including $x_0 = 0.2$ still undershoots, uniform $N = 10$ inflates user knots, PCHIP on $N = 5$ reduces to C^1). Right: restricted domain $[0.25, 0.85]$ (uniform knots) showing smooth interior agreement.

and satisfies $\min_i y_i \le y(t) \le \max_i y_i$. Setting $y_i \ge 0$ guarantees $y(t) \ge 0$ identically. □

Geometrically, $\mathbf{P}(t)$ admits De Casteljau's division-free recursive convex interpolation construction (Fig. 5):

$$\begin{aligned} \mathbf{Q}_i(t) &= (1-t)\mathbf{P}_i + t\mathbf{P}_{i+1}, && i \in \{0,1,2\}, \\ \mathbf{R}_i(t) &= (1-t)\mathbf{Q}_i(t) + t\mathbf{Q}_{i+1}(t), && i \in \{0,1\}, \\ \mathbf{P}(t) &= (1-t)\mathbf{R}_0(t) + t\mathbf{R}_1(t), \end{aligned}$$

where $\mathbf{R}_0(t)\mathbf{R}_1(t)$ gives the exact tangent direction and endpoints obey $\mathbf{P}'(0) = 3(\mathbf{P}_1 - \mathbf{P}_0)$ and $\mathbf{P}'(1) = 3(\mathbf{P}_3 - \mathbf{P}_2)$; `models.py::BezierProfile.get_curve` evaluates (8)–(9) directly over uniform t in $\mathcal{O}(1)$ per-point time, whereas physical-ROM queries in `solve_y`, `helpers.js::solveBezierY`, and `solver.js` run bounded bisection (at most 30 evaluations of $x(t)$ plus 1 final evaluation of $y(t)$, $\mathcal{O}(1)$ in control-point count).

While `BezierProfile.get_curve` evaluates (8)–(9) directly over $t \in [0, 1]$, querying tension at a physical ROM coordinate $x_{\text{target}} \in [x_0, x_3]$ requires inverting $x(t^*) = x_{\text{target}}$ (implemented in `models.py::BezierProfile.solve_y` and `helpers.js::solveBezierY`). Requiring $x_0 < x_1 < x_2 < x_3$ is a clean *sufficient* (though not necessary) condition for strict monotonicity: since $x'(t) = 3(1-t)^2(x_1 - x_0) + 6(1-t)t(x_2 - x_1) + 3t^2(x_3 - x_2)$ is 3 times a convex combination of strictly positive increments, $x'(t) > 0$ on $[0, 1]$ and t^* is unique. This invariant (together with $0 \le x_0 < x_1 < x_2 < x_3 \le 1$ and $y_i \ge 0$) is enforced at load and edit time in `BezierProfile.__post_init__`, `parser.py/parser.js` (`override:` handling), `WorkoutSolver` (`solver.js`), `solveBezierY`, and `BezierEditor.jsx`, and holds across all 61 exercises in

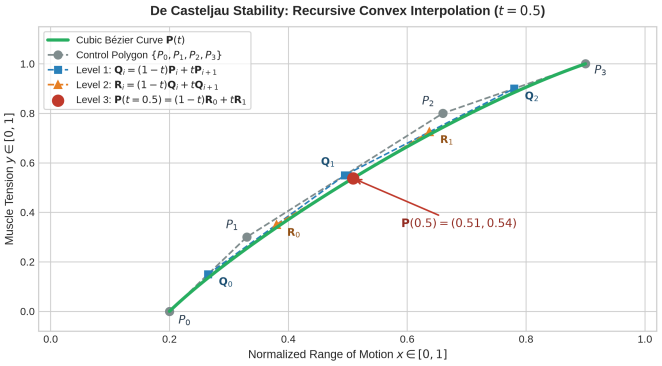


Figure 5: De Casteljau geometric construction at $t = 0.5$ on `LR`, requiring six division-free convex linear interpolations in $\mathcal{O}(1)$ time.

Table 1: Numerical properties of ROM tension approximations (per-point evaluation cost).

Method	**Params**	**Query Cost**	$y(x) \geq 0$	**Support** $[x_0, x_3]$
Lagrange (Equi.)	n nodes	$\mathcal{O}(n)$ (bary.)	No	Gibbs + Runge
Lagrange (CGL)	N nodes	$\mathcal{O}(N)$ (bary.)	No	Resting ripples
Natural C^2 Spline	N knots	$\mathcal{O}(\log N)$	No	Undershoot x_0
Cubic Bézier	**4 pts**	$\boldsymbol{\mathcal{O}(1)}$	**Yes**	**Exact** $[x_0, x_3]$

`defaultExercises.json`. After K bisection updates on $t \in [0, 1]$ (without early exit),

$$|t^{(K)} - t^*| \leq 2^{-(K+1)},$$
$$|x(t^{(K)}) - x_{\text{target}}| \leq 2^{-(K+1)} \max_{[0,1]} |x'(t)|. \tag{10}$$

Both `solve_y` and `solveBezierY` run up to $K = 30$ iterations, exiting early only when *both* $|x(t^{(k)}) - x_{\text{target}}| < 10^{-6}$ *and* the bracket width satisfies $u_k - \ell_k < 10^{-6}$. Because $t^{(k)}$ is the bracket midpoint, this dual criterion guarantees $|t^{(k)} - t^*| < 5 \times 10^{-7}$ and $|y(t^{(k)}) - y(t^*)| \leq \frac{1}{2} \cdot 10^{-6} \max_{[0,1]} |y'(t)|$ uniformly even when $x'(t^*)$ is arbitrarily small. Fig. 6, Fig. 7, and Table 1 summarize the Bernstein basis and method comparison.

3.4 Exact Parametric Quadrature vs. Trapezoidal Discretization

To conserve muscle contribution magnitude $M_{e,m}$ under arbitrary control-point shapes, consider integrating $y\,dx$ across $[x_0, x_3]$. Sampling $t_k = k/(R-1)$ ($R = 50$, $h = 1/(R-1)$) with $\Delta x_k = x(t_{k+1}) - x(t_k)$ via the composite trapezoidal rule gives

$$I_{\text{trap}} = \sum_{k=0}^{R-2} \frac{y(t_k) + y(t_{k+1})}{2} \Delta x_k. \tag{11}$$

Because t_k is uniform while $x(t_k)$ is *non-uniformly spaced*, the classical uniform-grid formula $-\frac{(x_3-x_0)^3}{12(R-1)^2} y''(\xi)$ does *not* apply. Under strict monotonicity ($x_0 < x_1 < x_2 < x_3$, ensuring $\Delta x_k > 0$ and $y \circ x^{-1} \in C^\infty([x_0, x_3])$), expanding $y(x)$ symmetrically around each spatial midpoint $m_k = \frac{1}{2}(x(t_k) + x(t_{k+1}))$ cancels all odd-degree terms in the single-interval error $e_k = \int_{x(t_k)}^{x(t_{k+1})} y(x)\,dx - \frac{y(t_k)+y(t_{k+1})}{2} \Delta x_k$, giving

$$E_{\text{trap}} = I_{\text{exact}} - I_{\text{trap}} = -\frac{1}{12} \sum_{k=0}^{R-2} y''(m_k)\,(\Delta x_k)^3 + \mathcal{O}\left(\sum_{k=0}^{R-2} (\Delta x_k)^5 \right), \tag{12}$$

with $y'' = d^2y/dx^2$. Since $m_k = x(t_{k+1/2}) + \mathcal{O}(h^2)$, $\Delta x_k = h x'(t_{k+1/2}) + \mathcal{O}(h^3)$, and $y''(x) = \big(y''(t)x'(t) - y'(t)x''(t)\big)/(x'(t))^3$, the midpoint Riemann sum has $\mathcal{O}(h^2)$ quadrature error and yields the $\mathcal{O}(h^2)$ parametric asymptotic expansion

$$E_{\text{trap}} = -\frac{h^2}{12} \int_0^1 \big(y''(t)x'(t) - y'(t)x''(t)\big)\,dt + \mathcal{O}(h^4), \tag{13}$$

which equals 2.31×10^{-5} on `LR` ($R = 50$). By contrast, applying a uniform-x trapezoidal rule on $[0, 1]$ integrates across the boundary discontinuity at x_3 on `LR` (and across jump discontinuities at both x_0 and x_3 on `Deadlift/RDL` *Erectors*), degrading convergence to $\mathcal{O}(h)$ with grid-alignment sensitivity: at $R = 50$ ($\Delta x = 1/49$), the relative error is 2.06% on `LR` and 1.97% on `Deadlift/RDL` *Erectors*, jumping to 2.51% and 9.52% at $R = 51$ ($\Delta x = 1/50 = 0.02$, matching `DashboardTab.jsx`'s step size where $x = 0.4, 0.6, 0.9$ align directly with grid nodes). Because $g(t) = y(t)x'(t)$ is a degree-5 polynomial on $[0, 1]$, 3-point Gauss–Legendre quadrature integrates $I_{\text{exact}} = \int_0^1 y(t)x'(t)\,dt$ *exactly* to machine precision ($2(3) - 1 = 5$):

$$I_{\text{GL3}} = \sum_{k=1}^{3} w_k\, y(t_k)\, x'(t_k), \qquad t_k = \frac{1 + \xi_k}{2},\ w_k = \frac{\tilde{w}_k}{2}, \tag{14}$$

with standard nodes $\xi_k \in \{-\sqrt{3/5}, 0, \sqrt{3/5}\}$ and weights $\tilde{w}_k \in \{5/9, 8/9, 5/9\}$. For each $m \in \mathcal{M}_e$, the area-normalized tension density is defined on $[0, 1]$ as

$$B_{e,m}(x) = \begin{cases} \dfrac{M_{e,m}}{I_{\text{GL3},e,m}}\, y_{e,m}\big(x_{e,m}^{-1}(x)\big), & x \in [x_{0,e,m}, x_{3,e,m}] \text{ and } I_{\text{GL3},e,} \\ 0, & \text{otherwise,} \end{cases} \tag{15}$$

and $B_{e,m}(x) \equiv 0$ for $m \notin \mathcal{M}_e$. `BezierProfile.exact_area()`, `helpers.js::bezierArea`, `solver.js`, and `DashboardTab.jsx` (which samples 51 ROM points $i/50$ versus 50 points $i/49$ in the solver) all evaluate (15), eliminating quadrature drift across both backend and frontend.

4 Physiological Dynamics & Optimization

4.1 Effective Volume, Dynamic TUT, and CNS Fatigue

In `constants.py` and `models.py` (`SetData`), each set contains n_b base repetitions, n_a assisted repetitions

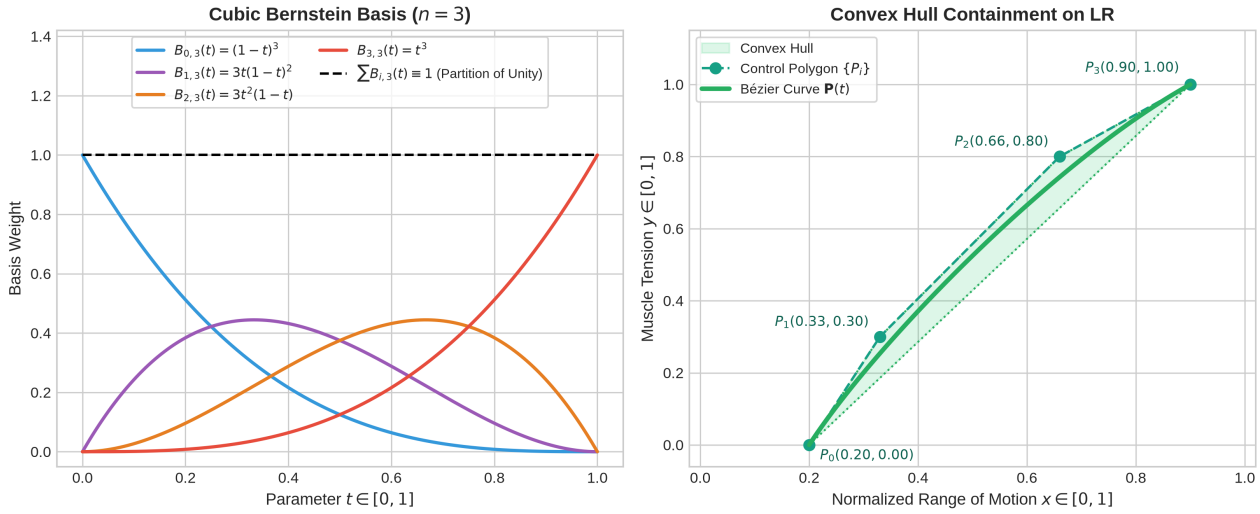


Figure 6: Left: Cubic Bernstein basis polynomials $B_{i,3}(t)$ verifying $\sum_{i=0}^{3} B_{i,3}(t) \equiv 1$. Right: Resulting `LR` curve enclosed within the convex hull of its four control points (with endpoints $\mathbf{P}_0, \mathbf{P}_3$ on the hull boundary).

($\alpha_{\text{ass}} = 0.5$), and n_p partial repetitions ($\alpha_{\text{part}} = 0.33$). Both `parser.py/parser.js` and `metrics.py` clamp RPE to $[0.0, 10.0]$ (defaulting omitted set RPE to 9.0, with `SetData` rejecting negative repetition counts or RPE < 0); by contrast, the simulated-annealing solver in `solver.py` restricts candidate RPE values to $[5.0, 10.0]$ during mutation as a search-space restriction. Total and HTMU-effective repetitions are:

$$n_{b,\text{eff}} = \min\big(n_b, \max(0, \text{RPE} - 4.0)\big), \tag{16}$$

$$R_{\text{tot}} = n_b + \alpha_{\text{ass}} n_a + \alpha_{\text{part}} n_p, \tag{17}$$

$$R_{\text{eff}} = n_{b,\text{eff}} + \alpha_{\text{ass}} n_a + \alpha_{\text{part}} n_p. \tag{18}$$

Base repetitions recruit high-threshold motor units when RIR < 6 (RPE > 4.0). Note that logbook-level summaries (`SetData`, `metrics.py`) credit $\alpha_{\text{ass}} n_a$ and $\alpha_{\text{part}} n_p$ unconditionally at the set level, whereas the mesocycle solver (§4.2) specifically restricts n_p to stretched-ROM partials on $[0, 0.3333]$ with RPE ≥ 9.0.

In `metrics.py` (`calculate_set_tuts`), given execution tempo (c, p_s, e, p_l) (default 1-0-2-0 s) with nominal duration $\tau_0 = c + p_s + e + p_l$, each base repetition $j \in \{0, \ldots, n_b - 1\}$ experiences instantaneous $\text{RIR}_j = (10.0 - \text{RPE}) + (n_b - 1 - j)$ and concentric slowdown $\Delta c(\text{RIR}_j)$:

$$\Delta c(\text{RIR}_j) = \begin{cases} 1.60\,\text{s}, & \text{RIR}_j < 1.0, \\ 1.00\,\text{s}, & 1.0 \leq \text{RIR}_j < 2.0, \\ 0.60\,\text{s}, & 2.0 \leq \text{RIR}_j < 3.0, \\ 0.35\,\text{s}, & 3.0 \leq \text{RIR}_j < 4.0, \\ 0.15\,\text{s}, & 4.0 \leq \text{RIR}_j < 5.0, \\ 0.00\,\text{s}, & \text{RIR}_j \geq 5.0, \end{cases} \tag{19}$$

yielding per-rep TUT $\tau_j = \tau_0 + \Delta c(\text{RIR}_j)$ for base reps, τ_0 for assisted reps, and $0.5\tau_0$ for partial reps. Set `total_tut` and `effective_tut` are computed during logbook ingestion in `parser.py::analyze_workout_log` (and on the fly in `metrics.py::calculate_metrics` when omitted), where `effective_tut` sums continuously over the last $\lfloor n_{b,\text{eff}} \rfloor$ base reps plus $\{n_{b,\text{eff}}\}\,\tau_{n_b - \lfloor n_{b,\text{eff}} \rfloor - 1}$ (when fractional remainder $\{n_{b,\text{eff}}\} > 0$) and unscaled assisted and partial rep durations (τ_0 and $0.5\tau_0$, unlike R_{tot}, R_{eff}, and Φ_{set} which apply α_{ass} and α_{part}).

In `metrics.py` (`calculate_set_fatigue`), per-rep effort $\text{RPE}_j = \max(0, 10.0 - \text{RIR}_j)$ drives a continuous exponential CNS fatigue multiplier $\mu(\text{RPE}_j) = 1.1^{\text{RPE}_j - 7.5}$ for all $\text{RPE}_j \geq 0$, while past-failure assisted and partial repetitions take the concentric-failure multiplier $1.1^{10.0 - 7.5} = 1.1^{2.5}$:

$$\Phi_{\text{set}} = F_{\text{ex}} \Bigg[\sum_{j=0}^{n_b - 1} \tau_j \, 1.1^{\text{RPE}_j - 7.5} + \alpha_{\text{ass}} 1.1^{2.5} n_a \tau_0 + 0.5\, \alpha_{\text{part}} 1.1^{2.5} n_p \tau_0 \Bigg], \tag{20}$$

where $F_{\text{ex}} = f_{\text{base}}$ (note that c_{load} scales weighted tonnage, not Φ_{set}, preventing low-c_{load} heavy compounds like `Squat` and `Deadlift` from scoring lower fatigue than light isolations). All set-level volume, tonnage (LR_{tot}), weighted tonnage ($LR_{\text{tot}} c_{\text{load}}$), TUT, and fatigue totals are projected onto queried macro- or sub-muscles via $M_{e,m}$ in `metrics.py::calculate_metrics`.

4.2 Constrained Multi-Objective Combinatorial Solver

In `solver.py` (`WorkoutSolver`), a candidate mesocycle $\omega \in \Omega$ assigns $D \geq 1$ training days, where each day ω_d is an ordered list of $N_d \in [N_{\min}, N_{\max}]$ ($N_{\min} \geq 1$) scheduled exercise slots (d, k), each with $S_{d,k} \in [S_{\min}, S_{\max}]$ ($S_{\min} \geq 1$) sets, per-set base reps $r \in [r_{\min}, r_{\max}]$ ($r_{\min} \geq 1$, with `add_ex/add_set`

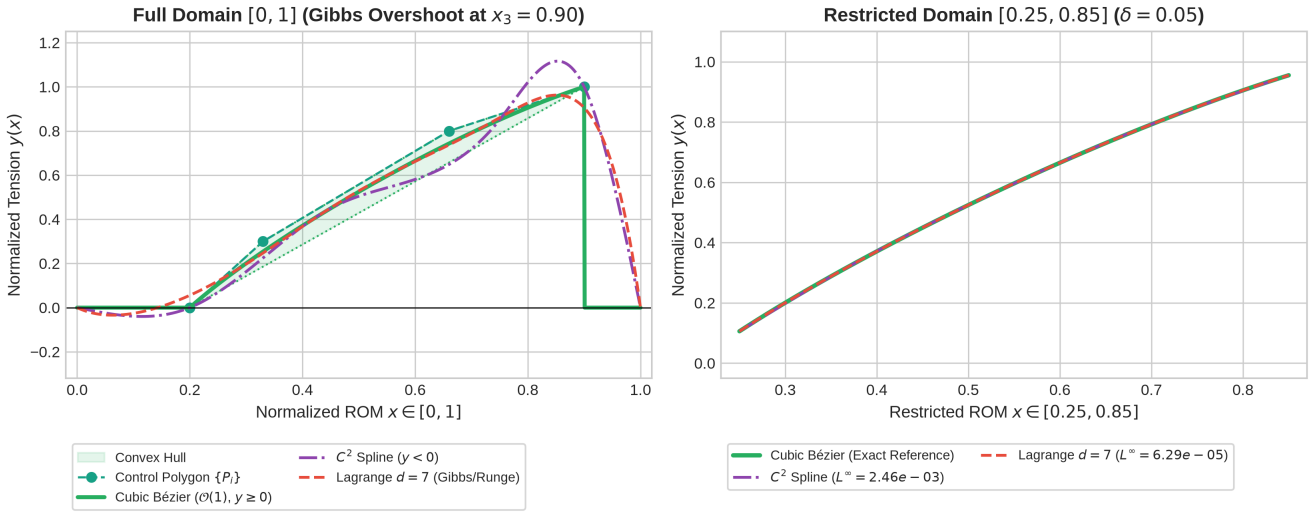


Figure 7: Synoptic comparison on `LR`. Left: full domain $[0, 1]$, showing Gibbs overshoot at $x_3 = 0.90$ for continuous global interpolants. Right: restricted domain $[0.25, 0.85]$ ($\delta = 0.05$), isolating smooth interior errors ($L^\infty = 6.29 \times 10^{-5}$ for Lagrange $d = 7$, 2.46×10^{-3} for $N = 6$ C^2 spline).

clamping the default 10 reps to $[r_{\min}, r_{\max}]$), RPE initialized in $\{7.0, 8.0, 8.5, 9.0, 10.0\}$ (8.0 on `add_ex`/`add_set`) and mutated in 0.5 increments within $[5.0, 10.0]$, and stretched-ROM partials $p \in \{0, 2, 4, 6\}$ (enabled only when RPE ≥ 9.0, cleared if mutation lowers RPE < 9.0). Let $x_i = i/(R-1)$, $i = 0, \ldots, R-1$ ($R = 50$), be a uniform grid on physical ROM $[0, 1]$, with $B_{e,m}(x_i)$ evaluated via `BezierProfile.get_physical_curve`. Writing $e = e(d, k)$ for the exercise at slot (d, k) and $N_{b,d,k} = \sum_s n_{b,d,k,s}$, $N_{p,d,k} = \sum_s n_{p,d,k,s}$ for slot-level rep sums over sets, the aggregate tension curve for muscle m is

$$V_m(x_i) = \sum_{d=1}^{D} \sum_{k=1}^{N_d} B_{e,m}(x_i)\big(N_{b,d,k} + \alpha_{\text{part}} N_{p,d,k} \chi_{\text{part}}(x_i)\big), \tag{21}$$

where $\chi_{\text{part}}(x_i) = \mathbb{I}[x_i \leq 0.3333]$ restricts partial repetitions to the stretched third of physical ROM and $\alpha_{\text{part}} = 0.33$ matches the partial weighting used in muscle volume totals.

Fast SA Surrogates. During Simulated Annealing search, `evaluate_workout` retains exact dynamic TUT while replacing load and fatigue with lightweight surrogates:

- *Load:* $\hat{L}_{d,k,s} = \frac{100.0}{1.0 + 0.033\left(n_{b,d,k,s} + \max(0, 10 - \text{RPE}_{d,k,s})\right)} \cdot c_{\text{load},e(d,k)}$ (RPE-aware 100 kg normalized 1RM Epley load weighted by articular efficiency c_{load}, avoiding logbook $\mu_{\text{load}}/q_{\text{load}}$ input-unit distortion).
- *TUT:* lightweight RIR-slowdown dynamic TUT matching §4.1 at default 1-0-2-0 s tempo ($\tau_0 = 3.0$ s, partials $0.5\tau_0$), keeping E_{tut} genuinely distinct from E_{vol}.
- *Fatigue:* linear $R_{\text{tot}} f_{\text{base}}$, omitting exponential RPE scaling and dynamic TUT (preserving exercise fatigue ordering for fixed set parameters, while treating daily balance F_d as RPE-invariant).

When target distribution shares p_m^{target} are configured, we evaluate $\mathcal{M}_{\text{eval}} = \{m : p_m^{\text{target}} > 0\}$ with simplex weights $\omega_m = p_m^{\text{target}}$ so untargeted synergists cannot dilute the curve objective; otherwise $\mathcal{M}_{\text{eval}} = \{m : \max_r V_m(x_r) > 0\}$ with ROM-active volume weights $\omega_m = W_m / \sum_{m' \in \mathcal{M}_{\text{eval}}} W_{m'}$ (or $1/|\mathcal{M}_{\text{eval}}|$ if $\sum_{m' \in \mathcal{M}_{\text{eval}}} W_{m'} = 0$). Writing $\hat{V}_m(x_i) = V_m(x_i)/\max_r V_m(x_r)$ when $\max_r V_m(x_r) > 0$ (and $\hat{V}_m \equiv 0$ otherwise), seven dimensionless error metrics $\mathcal{I} = \{\text{curve}, \text{vol}, \text{ton}, \text{tut}, \text{distr}, \text{variety}, \text{balance}\}$ evaluate mesocycle quality:

$$E_{\text{curve}} = \sum_{m \in \mathcal{M}_{\text{eval}}} \frac{\omega_m}{R} \sum_{i=0}^{R-1} \big(\hat{V}_m(x_i) - T_m(x_i)\big)^2, \tag{22}$$

$$E_q = \left(\frac{X_{q,\text{eff}}}{X_{q,\text{tot}}} - \rho_q^*\right)^2, \qquad q \in \{\text{vol}, \text{ton}, \text{tut}\}, \tag{23}$$

$$E_{\text{distr}} = \begin{cases} \frac{1}{2} \sum_m \big(p_m^{\text{actual}} - p_m^{\text{target}}\big)^2, & \text{if targets set,} \\ 0, & \text{otherwise,} \end{cases} \tag{24}$$

$$E_{\text{variety}} = \left(1 - \frac{|\mathcal{U}(\omega)|}{N_{\text{tot}}(\omega)}\right)^2, \tag{25}$$

$$E_{\text{balance}} = \begin{cases} \frac{1}{2}\left(\frac{\text{Var}_d(F_d)}{(F_{\max}/2)^2} + \frac{\text{Var}_d(V_d)}{(V_{\max}/2)^2}\right), & D > 1, \\ 0, & D \leq 1, \end{cases} \tag{26}$$

where $T_m(x)$ defaults to `constant` (1) for unconfigured muscles and selects among `constant` (1), `linear` $(1 - x)$, and peak-normalized `sigmoid` / `inv_sigmoid` $S_\pm(x)/\max_{x'} S_\pm(x')$ with $S_\pm(x) = 1/(1 + e^{\mp 10(x - 0.5)})$ (expanded across macro sub-muscles in `MUSCLES` with explicit sub-muscle keys overriding macros, aligning $\max_x T_m(x) = 1$ with peak-normalized $\hat{V}_m \in [0, 1]$), $(X_{\text{vol,tot}}, X_{\text{vol,eff}}) = \sum_{d,k,s} (R_{\text{tot}}, R_{\text{eff}})$, $(X_{\text{ton,tot}}, X_{\text{ton,eff}}) = \sum_{d,k,s} \hat{L}_{d,k,s} (R_{\text{tot}}, R_{\text{eff}})$, and

$(X_{\text{tut,tot}}, X_{\text{tut,eff}}) = \sum_{d,k,s}(\text{TUT}_{\text{tot}}, \text{TUT}_{\text{eff}})$ with defaults $\rho^*_{\text{vol}} = \rho^*_{\text{ton}} = 0.70$ and $\rho^*_{\text{tut}} = 0.74$ (because RIR concentric slowdown increases late-rep duration and partials take $0.5\tau_0$ vs. $\alpha_{\text{part}} = 0.33$, every non-degenerate set satisfies $\text{TUT}_{\text{eff}}/\text{TUT}_{\text{tot}} \geq R_{\text{eff}}/R_{\text{tot}}$, with strict inequality whenever $n_b > \text{RPE} - 4.0$ and either $\text{RPE} > 5.0$ or $n_p > 0$; at $\rho^*_{\text{vol}} = 0.70$, the feasible TUT ratio spans $[0.7152, 0.7546]$ for $r \in [5, 20]$ and $[0.7202, 0.7522]$ for $r \in [6, 15]$, making $\rho^*_{\text{tut}} = \rho^*_{\text{vol}} = 0.70$ jointly infeasible while $\rho^*_{\text{tut}} = 0.74$ lies inside the jointly feasible strip), p_m^{target} is the simplex-normalized user-configured target share from `volume_dist` (falling back to uniform division across a macro's database sub-muscles—or across all `MUSCLES` sub-muscles of that macro if none appear in the database—when `subs` omits that macro), $p_m^{\text{actual}} = W_m / \sum_{m'} W_{m'}$ is the actual share of muscle-weighted ROM-active volume $W_m = \sum_{d,k}\big(M_{e,m}\mathbb{I}[I_{\text{GL3},e,m} > 0]N_{b,d,k} + \alpha_{\text{part}} M_{e,m,\text{part}} N_{p,d,k}\big)$, where $M_{e,m,\text{part}} = \int_0^{0.3333} B_{e,m}(x)\,dx$ is integrated in parameter space via 3-point Gauss–Legendre quadrature over $[0, t_{\text{cut}}]$ with $t_{\text{cut}} = 0$ for $0.3333 \leq x_{0,e,m}$, $t_{\text{cut}} = 1$ for $0.3333 \geq x_{3,e,m}$, and $t_{\text{cut}} = x^{-1}_{e,m}(0.3333)$ (50-step bisection to 10^{-12}) otherwise (matching $\int_0^1 V_m(x)\,dx$ in (21)), $\mathcal{U}(\omega)$ is the set of distinct scheduled exercise names, $N_{\text{tot}}(\omega) = \sum_{d=1}^{D} N_d$ is total scheduled slots, V_d and $F_d = \sum_{k=1}^{N_d}\sum_{s=1}^{S_{d,k}} R_{\text{tot},d,k,s}\, f_{\text{base},e(d,k)}$ denote daily total reps and linear fatigue, $\text{Var}_d(\cdot)$ denotes the population variance (`np.var`, $\text{ddof} = 0$) across days $d = 1, \ldots, D$, $V_{\max} = N_{\max} S_{\max}(r_{\max} + 6\alpha_{\text{part}})$, and $F_{\max} = \max(10, \max_e f_{\text{base},e})\, V_{\max}$ (noting Popoviciu's $(V_{\max}/2)^2$ denominator is conservative since $V_d \geq N_{\min} S_{\min} r_{\min} > 0$).

Prior to optimization, `calibrate()` estimates normalization denominators via the empirical sample mean $\hat{\mu}_i = \frac{1}{K}\sum_{r=1}^{K} E_i(\omega^{(r)})$ over $K = 500$ random states (`SolverConfig` default; `GeneratorConfig.jsx` UI defaults to $K = 1000$, Softmax-uniform weights $w_i = 1/7$, `inv_sigmoid` targets, and integer-rounded uniform macro shares across the 7 database-active macro groups subdivided equally within each macro's active sub-muscles, with $K = 0$ disabling calibration cleanly in both engines), setting $s_i = 2\hat{\mu}_i$ (with $\mathcal{O}(K^{-1/2})$ standard error, $\mathcal{O}(K^{-1})$ sampling variance, and zero-guard $s_i \leftarrow 10^{-6}$ applied inside `evaluate_workout` whenever $s_i = 0$). `solve()` then minimizes

$$\min_{\omega \in \Omega} J(\omega) = 10^4 \sum_{i \in \mathcal{I}} w_i \frac{E_i(\omega)}{s_i} + 10^3\, P_{\text{dup}}(\omega), \tag{27}$$

where $P_{\text{dup}}(\omega) = 150 \cdot N_{\text{dup}}(\omega)$ penalizes same-day duplicate exercises, using Simulated Annealing with eight discrete no-op-filtered mutation operators, unified initial temperature $T_0 = 50.0$, geometric cooling $T_{t+1} = \gamma T_t$ ($\gamma = (0.01/T_0)^{1/N_{\text{iter}}}$, applied after each step $t = 0, \ldots, N_{\text{iter}} - 1$ so $T_{N_{\text{iter}}} = 0.01$ at termination), and Metropolis acceptance $\exp(-\Delta J / T_t)$.

5 Conclusions & Software Architecture

Iron Stats replaces scalar training logs with Bernstein-basis Bézier tension fields having strictly monotone ROM coordinate $x(t)$, RIR-aware concentric slowdown, continuous exponential CNS fatigue, and Monte Carlo calibrated Simulated Annealing. The Python analyzer (`python_research/analyzer/`, packaged via `pyproject.toml` and `requirements.txt` with `numpy` as sole runtime dependency) and the React/Vite/Capacitor frontend (`frontend/`) share the exact same Bézier curve definition (7)–(9), 3-point Gauss–Legendre area normalization across `models.py`, `solver.js`, and `DashboardTab.jsx`, and the same solver objective (differing intentionally in the interactive SVG editor `BezierEditor.jsx` and exercise-modal previews `helpers.js::getBezierCurveData` and `ExerciseModal.jsx`, which normalize by peak height $y/\max y$ rather than area). Future work will collect longitudinal training datasets to empirically calibrate weights w_i and integrate IMU or computer-vision kinematics to refine exercise-specific $B_{e,m}(x)$ profiles.